DECA Lab Spring Term

Part3: EEP1 Pipelining and Interrupts

Department of Electrical and Electronic Engineering

Imperial College London

v2.0

Spring 2025

Contents

1 Introduction **1**

2 Glossary **2**

3 EEP1 Pipeline **3**

3.1 Understanding the EEP1P pipeline 3

3.2 Removing stalls in the EEP1 pipeline 3

4 EEP1 Interrupt Mechanism **5**

4.1 Using a polling loop to detect hardware status 5

4.2 Interrupt interface to hardware 5

4.3 Multiple interrupt sources 6

5 Challenge **6**

1 Introduction

Before the lab

- Download Spring Part3-2025 from EE1 labs repository. Lab3 contains two completed designs for EEP1 with interrupts and pipeline added respectively.
- Read Spring lectures 7 and 8.

In this lab you will use two new (complete) versions of EEP1. EEP1P has a pipelined architecture, while EEP1I includes additional external memory-mapped hardware together with interrupt support.

In Section 3 you will run a short program on EEP1P and observe how its state machine operates, including which instructions cause pipeline stalls. In Section 4 you will run programs illustrating how to interface to an external hardware timer, both with and without using interrupts.

In this lab you are not expected (except where explicitly stated) to understand the added hardware in detail. For EEP1I you are provided with a simplified schematic (Figure 5) that contains sufficient information to complete the exercises.

2 Glossary

Pipeline A processor organisation in which instruction execution is divided into stages so multiple instructions can be processed concurrently.

FETCH / EXEC1 / EXEC2 The three pipeline stages in EEP1P: FETCH reads the instruction memory from RAM, EXEC1 performs decode and part of execution, EXEC2 completes operations such as memory access.

Pipeline Register A register inserted between pipeline stages to hold intermediate values so that successive instructions can overlap in execution.

Pipeline Stall A delay inserted into pipeline execution when an instruction cannot proceed safely, typically due to data dependencies or resource conflicts.

Register Bypass (Forwarding) Hardware allowing a result to be routed directly from a later pipeline stage to an earlier stage without first writing it back to the register file.

State Machine Sequential logic whose outputs depend on current inputs and stored state; used in EEP1P control to coordinate pipeline stages.

Block RAM Dedicated synchronous memory blocks available inside FPGAs that often include built-in pipeline registers.

Interrupt A hardware mechanism that temporarily suspends normal program execution so a processor can respond to an external or internal event.

Interrupt Request (IRQ) A signal indicating that an interrupt source requires processor attention.

Interrupt Service Routine (ISR) A program executed automatically in response to an interrupt to service the event before returning to the main program.

Polling A software technique in which a program repeatedly checks hardware status instead of relying on interrupts.

Memory-Mapped I/O A technique where hardware control/status registers appear as memory locations accessible using normal load/store instructions.

INT0 / INT1 Interrupt sources provided in the lab hardware: INT0 is externally triggered; INT1 is a periodic timer interrupt.

SETI / CLRI Instructions that enable (SETI) or disable (CLRI) interrupt handling in the processor.

RETINT Instruction used to return from an interrupt service routine while restoring processor state.

PC (Program Counter) Register holding the address of the next instruction to be executed.

PCX A saved copy of the Program Counter used during interrupt handling.

N,Z,C,V Flags Processor condition flags: Negative, Zero, Carry, and Overflow.

Base Program The main program executing normally when no interrupt is being serviced.

FPGA Field-Programmable Gate Array; reconfigurable hardware capable of implementing digital circuits such as CPUs.

3 EEP1 Pipeline

Figure 1 shows the top-level sheet of EEP1P provided in the **ee1p** Issie project. Compared with EEP1, this adds a state machine (SM1) and two pipeline registers, INSREG and DOUTREG. These registers are placed after the instruction and data memories respectively to implement a three-stage pipeline: FETCH, EXEC1, and EXEC2, as shown in Figure 2.

The third stage (EXEC2) is required because of the DOUTREG pipeline register. Without this register, EXEC1 would need to perform an LDR instruction by reading the data RAM and writing to the register file within a single clock cycle. Introducing the third stage allows a higher clock frequency.

It is also worth noting that efficient FPGA synthesis of EEP1 (not required for this module, but possible using an Issie stick) normally requires both memories to be implemented as FPGA *block RAM*. These memories include built-in pipeline registers, unlike the asynchronous RAM used in Issie simulation, and therefore DOUTREG is required for a realistic implementation.

3.1 Understanding the EEP1P pipeline

Display the first nine clock cycles of the **ee1p** sheet of the **lab3p** design running the `testpipeline.ram` program in the waveform simulator. Configure the simulator to show INSREG.Q, DOUTREG.Q, CPENV, EXEC2V, INITV, together with the PC and register values. Refer to the `testpipeline.txt` file to see the simulated instruction sequence.

□ **Task 1.** *Load the **statemachine** sheet of **ee1p** to examine the design of SM1. The two flip-flops implement two independent state machines. Draw two separate state diagrams: one for FF1 and one for EXEC2FF.*

Figure 3 shows the pipeline stages for a program similar to `testpipeline.txt`. The STR (S2) instruction and the memory-interface signals for LDR and STR have been added at the appropriate points. Explain the different timing of these signals for LDR and STR.

Using your FF1 state diagram, explain the INITV waveform and describe its effect on clock cycle 0 in Figure 3. EXEC2V is high during EXEC2: determine from your EXEC2FF state diagram when this occurs and check that this matches Figure 3.

□ **Task 2.** *Examine MEMADDR on the **controlpath** sheet of **lab3p** and compare it with the unpipelined **lab2** EEP1 control path. Explain how the modification to MEMADDR allows the correct next instruction to be fetched. You may find it helpful to compare the data delays between PCNEXT and INS in the two designs.*

3.2 Removing stalls in the EEP1 pipeline

This section is non-examinable: it will not be assessed in lab interviews and is included for interest. You may return to it if you finish the main lab early.

Much modern CPU design focuses on techniques to avoid pipeline stalls. Can this be achieved in the EEP1P pipeline? The stall occurs because EXEC2 for an LDR instruction cannot overlap with EXEC1 of the following instruction. If such overlap were possible, the stall would disappear.

- The most obvious limitation is that EXEC1 and EXEC2 both use the same register-file write port, which can only write one register per cycle. This can be resolved relatively easily by adding a second write port, as discussed in Lecture 2.

- A subtler issue arises because, if EXEC2 of an LDR overlaps with EXEC1 of the next instruction, the destination register of the LDR has not yet been written when EXEC1 reads it. This can be solved with extra hardware: each EXEC1 register-file read port feeds one input of a MUX, with the other input driven directly from DOUTREG. When EXEC2 is active and EXEC1 attempts to read the register being written by EXEC2, the MUX selects the forwarded DOUTREG value. This *register bypass logic* is sufficient to eliminate the stall.

- A final issue is that the existing 16-bit DOUTREG pipeline register holds only the memory data output. To overlap EXEC2 and EXEC1, additional flip-flops are needed to store the information required for EXEC2 control: specifically the destination register number for an LDR instruction and an indication that an EXEC2 write must occur. This requires four extra bits, increasing the EXEC1–EXEC2 pipeline register width to 20 bits.

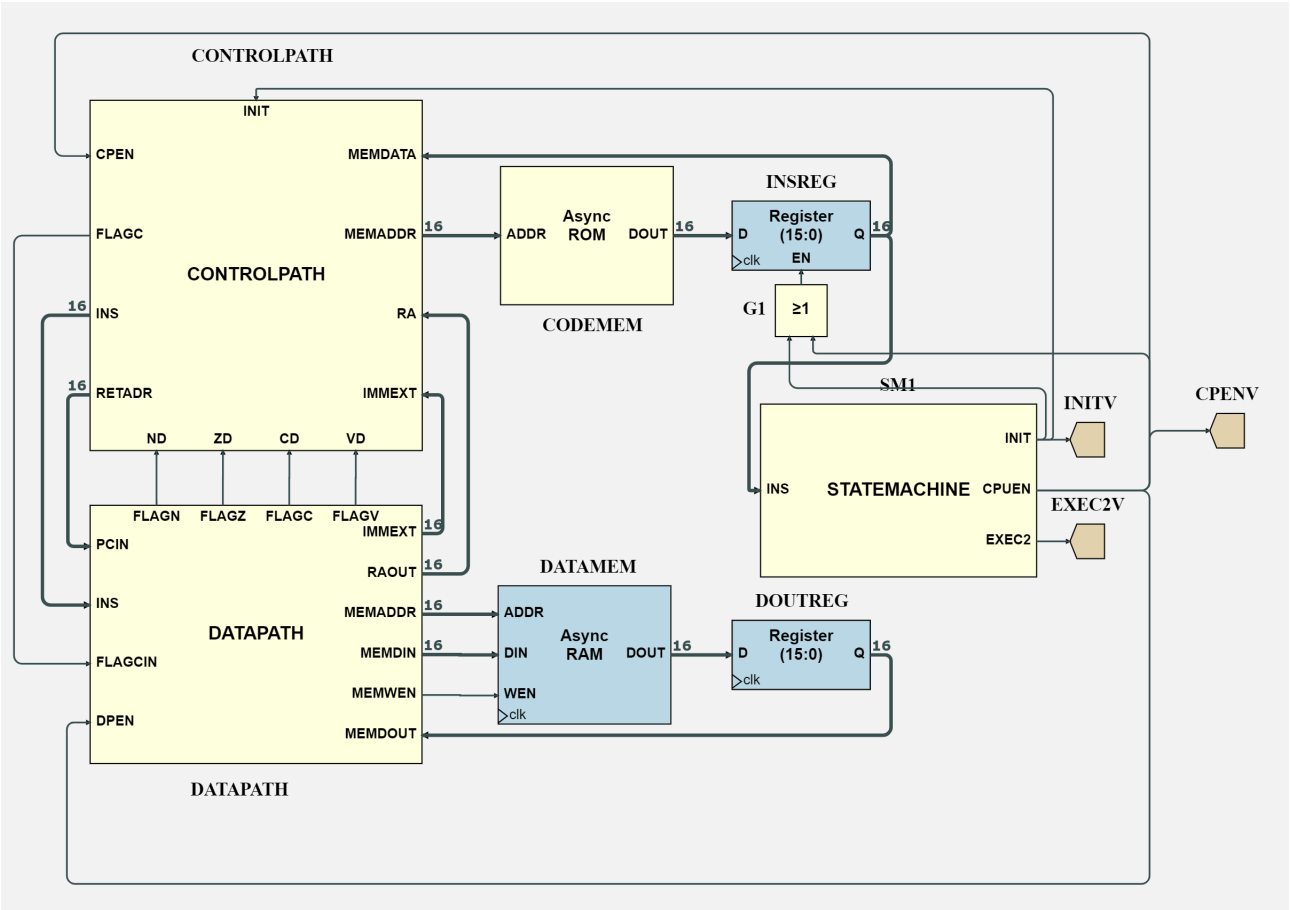

Figure 1: EEP1P schematic

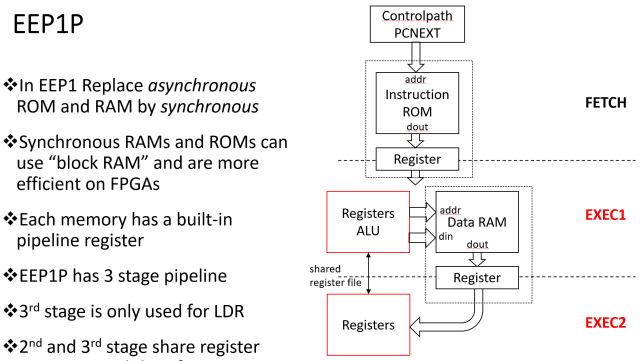

Figure 2: EEP1P pipeline description

Machine cycle →	0	1	2	3	4	5	6	7
	N => not LDR or STR, L => LDR, S => STR							
FETCH	N1	N2	L3	N4	N4	S5	N6	
EXEC1		N1	N2	L3		N4	S5	N6
EXEC2					L3			
ADDR, WEN				L3			S5	
DIN							S5	
DOUT					L3			

Figure 3: EEP1P pipeline diagram

With these additions — a second register-file write port, register-file bypass logic on both read ports, and an expanded EXEC2 pipeline register holding the destination register identifier and write-enable control — the pipelined EEP1 can operate without stalls.

4 EEP1 Interrupt Mechanism

This section explores the additional interrupt hardware provided in EEP1I and how it can be used. To support interrupts, the EEP1I CPU (provided in the eep1i project) includes three additions relative to EEP1:

- New hardware (shown in red in the control path, Figure 5) implementing an *interrupt mechanism*.
- Additional instructions (Figure 7) allowing read/write access to the `N,Z,C,V` flags, interrupt-related registers, and return from an interrupt service routine. These reuse unused encodings of EEP1 register-operand MOV instructions. For standard register operand MOV instructions, R_c is unused and therefore normally $c = \texttt{INS}(4{:}2) = 0$; the new instructions use encodings where $c \neq 0$.
- External hardware on sheet externalhardware, summarised in Figure 5. This provides two interrupt sources interfaced to EEP1 as shown in Figure 4:
  - **Interrupt 0 (INT0):** an interrupt request is generated whenever input `IRQ0TOGGLE` is toggled in the step simulator. To test IRQ0, the EEP1 CPU (`eep1`) must be placed in a top-level sheet (`eep1x`) that drives this input. The supplied `eep1x` design toggles `IRQ0TOGGLE` once every 128 cycles during simulation.
  - **Interrupt 1 (INT1):** a periodic interrupt generated by a counter approximately every 20 clock cycles.

4.1 Using a polling loop to detect hardware status

This section examines the external interrupt-generating hardware. Interrupt 0 (INT0) originates from the **externalhardware** sheet. An *interrupt request* (IRQ0) is set whenever the `IRQ0TOGGLE` input changes state. INT0 can be cleared by EEP1 instructions that write to the `Reset INT0` bit in memory-mapped I/O (Figure 4).

The program `iotest1.txt` uses the interrupt-status and reset bits from Figure 4 to detect INT0. The program repeatedly reads the IRQ0 status bit to determine whether the interrupt source is active. This technique is known as *polling* and contrasts with the interrupt-driven approach used in the next section.

□ **Task 3.** *Simulate the **eep1** (not **eep1x**) sheet of **lab3i** with instruction memory loaded from `iotest1.ram` in the **step simulator**. Run a few cycles to observe the normal polling loop. Then manually toggle `IRQ0TOGGLE` and observe how this generates an IRQ0 request, which the program detects and clears, incrementing R2. The `iotest1.txt` program therefore counts edges on `IRQ0TOGGLE` using memory-mapped I/O and a software loop.*

Repeat the task using `iotest2.ram` in the waveform simulator. The program `iotest2.txt` detects interrupt source 1 (INT1), which repeatedly sets IRQ1 approximately every 20 clock cycles via a counter on the **externalhardware** sheet. View IRQ1 and IRQ1reset in addition to PC, INS, R0, and R2. The code is otherwise identical to `iotest1.txt`, so you should observe IRQ1 being detected and reset at this regular interval.

4.2 Interrupt interface to hardware

The program `int-test1.txt` contains code starting at address 0 that enables interrupts (`SETI`) and then loops continuously incrementing R1. This loop is the *base program*. An Interrupt Service Routine (ISR), located at address 0xFF00, services the INT1 timer interrupt.

Each time IRQ1 is asserted, EEP1I automatically jumps to the ISR at 0xFF00. The ISR resets the interrupt, increments R2, and then returns to the base program using `RETINT`, restoring the flags so that base program execution is unaffected. The interrupt vector address (0xFF00) is fixed in the EEP1I hardware.

□ **Task 4.** *Simulate the **eep1** sheet of **lab3i** running `int-test1.ram` in the **waveform simulator**. View IRQ1, IRQ1reset, NZCVout, and PCX in addition to PC, INS, R0, and R2. Trace execution from the moment IRQ1 is asserted through the interrupt jump, ISR execution, and the `RETINT` return to the base program.*

For each observed waveform change, determine whether it is caused by an instruction or by the interrupt mechanism. Using the code and waveforms, explain why the base program PC, flags, and R0 remain unaffected by ISR execution.

4.3 Multiple interrupt sources

The program `int-test2.txt` demonstrates handling two interrupt sources simultaneously. Both sources are ORed onto the single interrupt request input (IRQ) of the **eep1** sheet. When an interrupt occurs, the ISR software determines which source is active, clears it, performs the appropriate action, and returns.

In this test, `IRQ0TOGGLE` is driven by a counter in the top-level `eep1x` sheet to generate an INT0 interrupt approximately every 128 cycles, starting at cycle 0.

□ **Task 5.** *Simulate the* ***eep1x*** *(not* ***eep1****) sheet of* ***lab3i*** *running* ***`int-test2.ram`*** *in the waveform simulator. Refer to the assembler listing in* ***`int-test2.txt`****. View FFIEN, IRQ0, IRQ0reset, IRQ1, IRQ1reset, NZCVout, and PCX in addition to PC, INS, R0, R2, and R3.*

From the first 50 clock cycles determine which cycles execute the base program, which execute INT0 servicing, and which execute INT1 servicing. Confirm that after each interrupt return, R0 and the N,Z,C,V flags remain unchanged.

□ **Task 6.** *Simulate the* ***eep1x*** *sheet of* ***lab3i*** *running* ***`int-test3.ram`*** *in the waveform simulator, referring to the assembler in* ***`int-test3.txt`****.*

- *Note the placement of* ***`SETI`*** *and* ***`CLRI`*** *inside the INT0 ISR. What do these instructions control? Explain why correct nesting of INT1 inside INT0 requires both instructions, and why* ***`SETI`*** *cannot be placed earlier.*
- *Explain why PCX must be saved and later restored in the INT0 ISR here, whereas this was not necessary in* ***`int-test2.txt`****.*

5 Challenge

The interrupt mechanism in EEP1I is deliberately minimal: it provides just enough hardware to allow software to save and restore the entire CPU state during an interrupt. As you have seen in this lab, this results in relatively complex and time-consuming interrupt software.

For this challenge you will design an extended version of EEP1, called EEP1S, that adds specialised hardware support for *re-entrant interrupts* using a stack. You should retain the external interrupt hardware developed in this lab, but replace the EEP1 CPU core with your new EEP1S design. The existing test programs may then be adapted to validate your implementation.

A possible design might include the following elements (you are free to modify or improve this proposal):

- A new 16-bit register SP (Stack Pointer) holding the address of a memory region used as an interrupt stack.
- New MOVC-style instructions to move values to and from SP.
- Instructions to push PC and FLAGS onto the stack using SP.
- Instructions to pop PC and FLAGS from the stack.

This challenge tests understanding at several levels. The work can be divided into separate components as enumrated below, which may be attempted independently. Students are encouraged to collaborate, with different members focusing on different components.

1. A precise ISA specification of the new features. In particular, you must define:
   - The exact semantics of each new instruction.
   - Whether and how flags are affected.
   - The structure of the stack (e.g. growth direction, full or empty, position in memory).
   - A precise description of interrupt entry and exit using the new instructions together with the interrupt hardware.

   The ISA specification must be fully precise: even small, unintended state changes during interrupt entry or exit can break re–entrant interrupt behaviour.
2. Hardware modifications required to implement the ISA (the EEP1S design).
3. Software to test and validate the new interrupt mechanism.
4. A formal analysis of the proposed ISA and interrupt entry/exit mechanism, demonstrating that re-entrant interrupts operate correctly under all circumstances (including nested interrupts and edge cases).

Address	Read			Write			Read Operation	Write Operation
	15:2	1	0	15:2	1	0		
0xFFFF	COUNT			n/a	Reset INT1	Reset INT0	Read INT1 counter value	1 => reset the corresponding interrupt request (IRQ1 or IRQ0)
0xFFFE	0	IRQ1	IRQ0				Read interrupt request status: 1 => on	n/a

Figure 4: EEP1I I/O Memory map

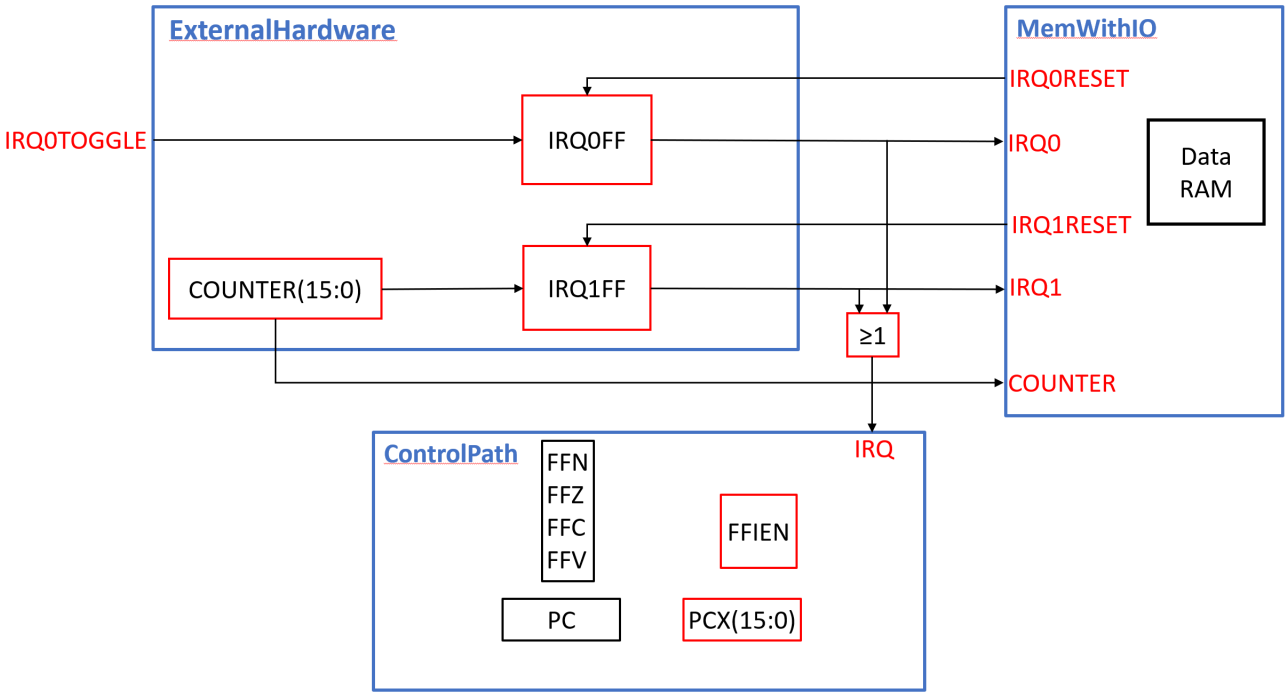

Figure 5: EEP1I Additional Hardware for Interrupts

Name	Waveform	Function
PCX	PCX.Q(15:0)	Temporary storage for return address from interrupt
FlagI	FFIEN.Q	CPU interrupt enable
IRQ0FF	IRQ0FF.Q	Interrupt 0 request (IRQ0): set when IRQ0Toggle input is toggled
IRQ1FF	IRQ1FF.Q	Interrupt 1 request (IRQ1): set at regular intervals (every 20 clock cycles)
IRQ0RESET	IRQ0RESET	Reset for Interrupt 0
IRQ1RESET	IRQ1RESET	Reset for interrupt 1
NZCVOUT	NZCVOUT(15:0)	The 4 EEP1 flags, combined as bits (3:0) of a 16 bit word.

Figure 6: EEP1P Interrupt control registers and flip-flops

EEP1 "Extra MOV" Coding	INS (8)	C (4:2)	A (11:9)	B (7:5)	INS	Alternative Mnemonic	Operation	Notes
MOV R6, R0, R0	0	6	0	0	0x0018	**RETINT**	PC := PCX; FlagI := 1	*Return from interrupt, enable interrupts*
MOV R6, R0, R1	0	6	0	1	0x0038	**SETI**	FlagI := 1	*Allow (enable) interrupts*
MOV R6, R0, R2	0	6	0	2	0x0058	**CLRI**	FlagI := 0	*Disable interrupts*
MOV R7, Ra, R0	0	7	a	0		**MOV Flags, Ra**	NZCV := Ra(3:0)	*Restore flags from register (no write to Ra)*
MOV R7, Ra, R1	0	7	a	1		**MOV PCX, Ra**	PCX := Ra	*Restore PCX from register (no write to Ra)*
MOV R7, Ra, R2	0	7	a	2		**MOV Ra, Flags**	Ra(3:0) := NZCV	*Save flags to register*
MOV R7, Ra, R3	0	7	a	3		**MOV Ra, PCX**	Ra := PCX	*Save PCX to register*

Figure 7: EEP1I Instructions to support interrupts